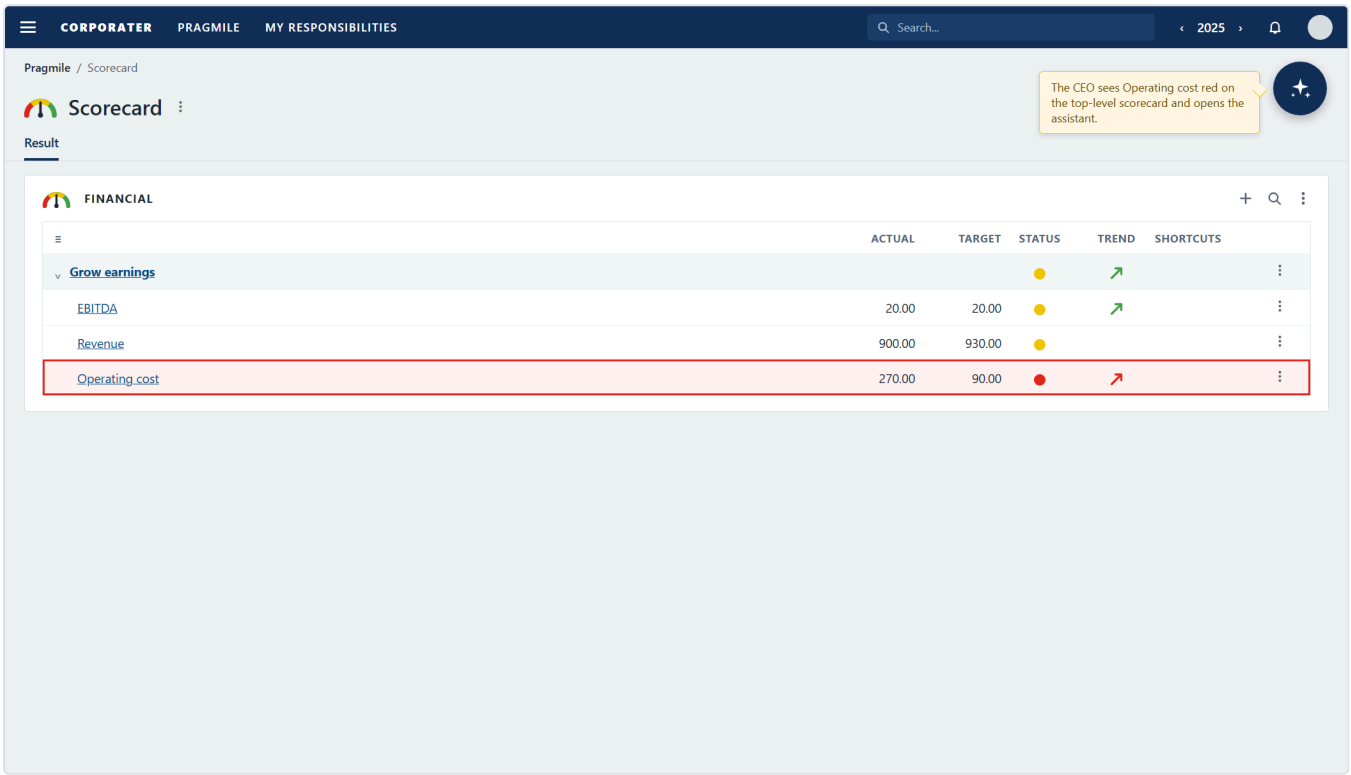
BMP AI Dashboarding — MVP scenario and solution flow

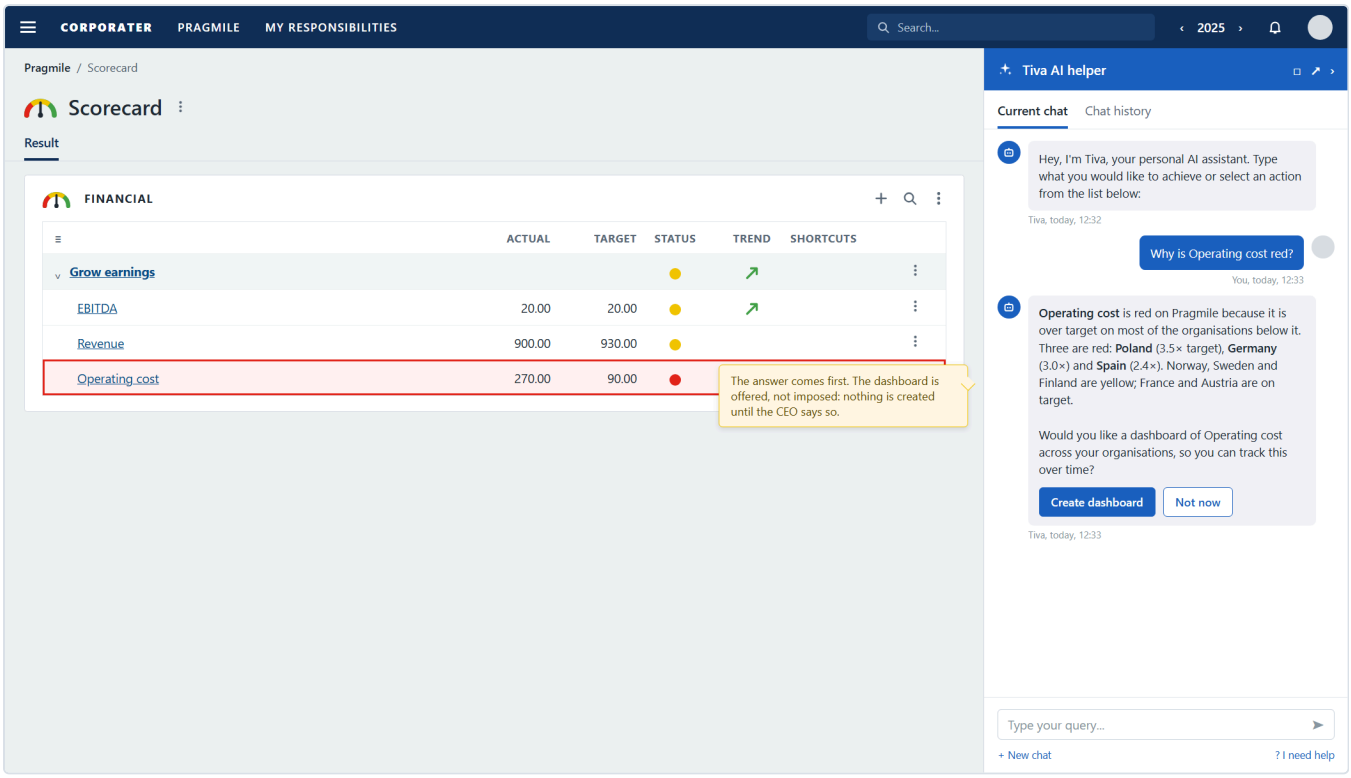
Draft from the Analytics team, 18 Sep 2026. Audience: implementation teams in Bedrock, Analytics, Agent Orchestration, Web and Dashboard. Builds on the Solution Overview of 20 Aug 2026 and replaces its section 3 for the MVP. Items marked **TBD** are not settled.

1. The scenario, as the user sees it

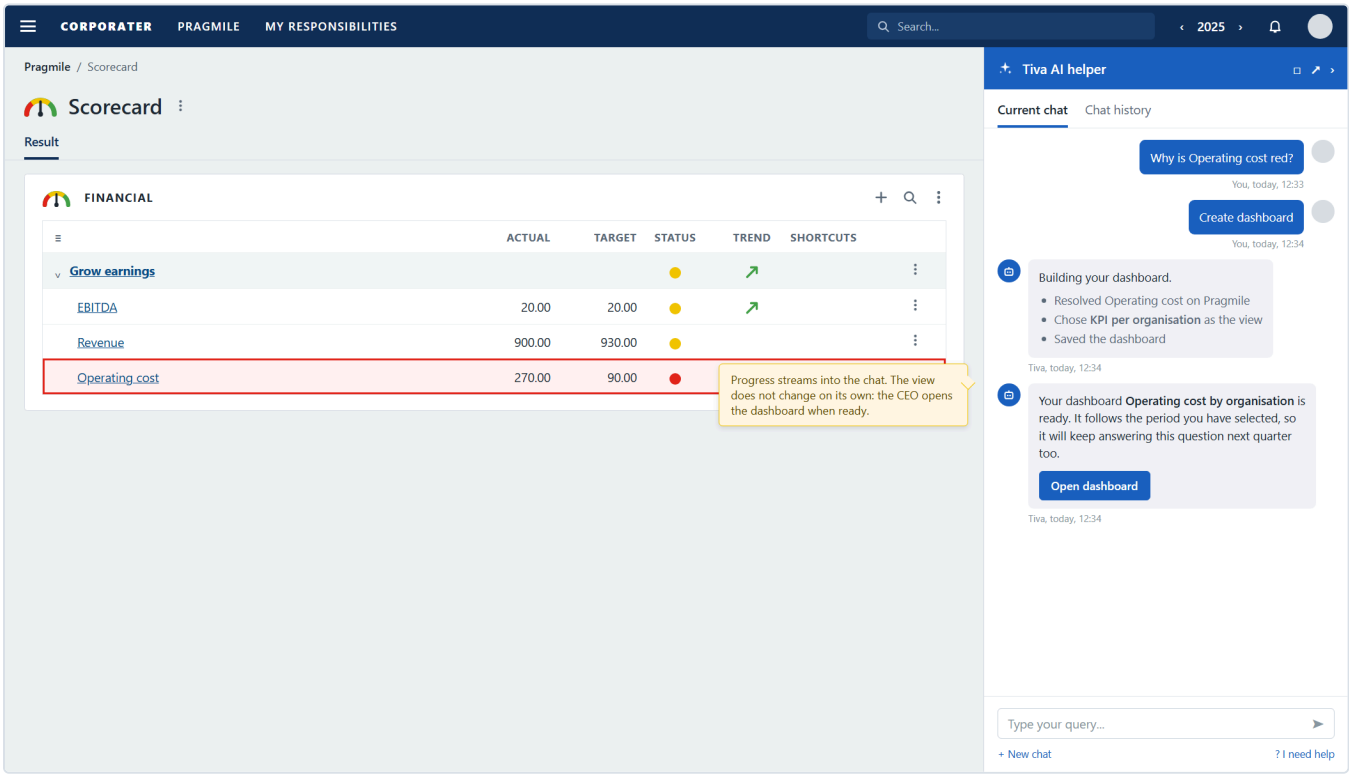
A CEO looks at the top-level scorecard and sees Operating cost red. They ask the assistant why. The assistant answers in words, then offers a dashboard that keeps answering the question as periods move. Nothing is created and no view changes until the CEO says so.



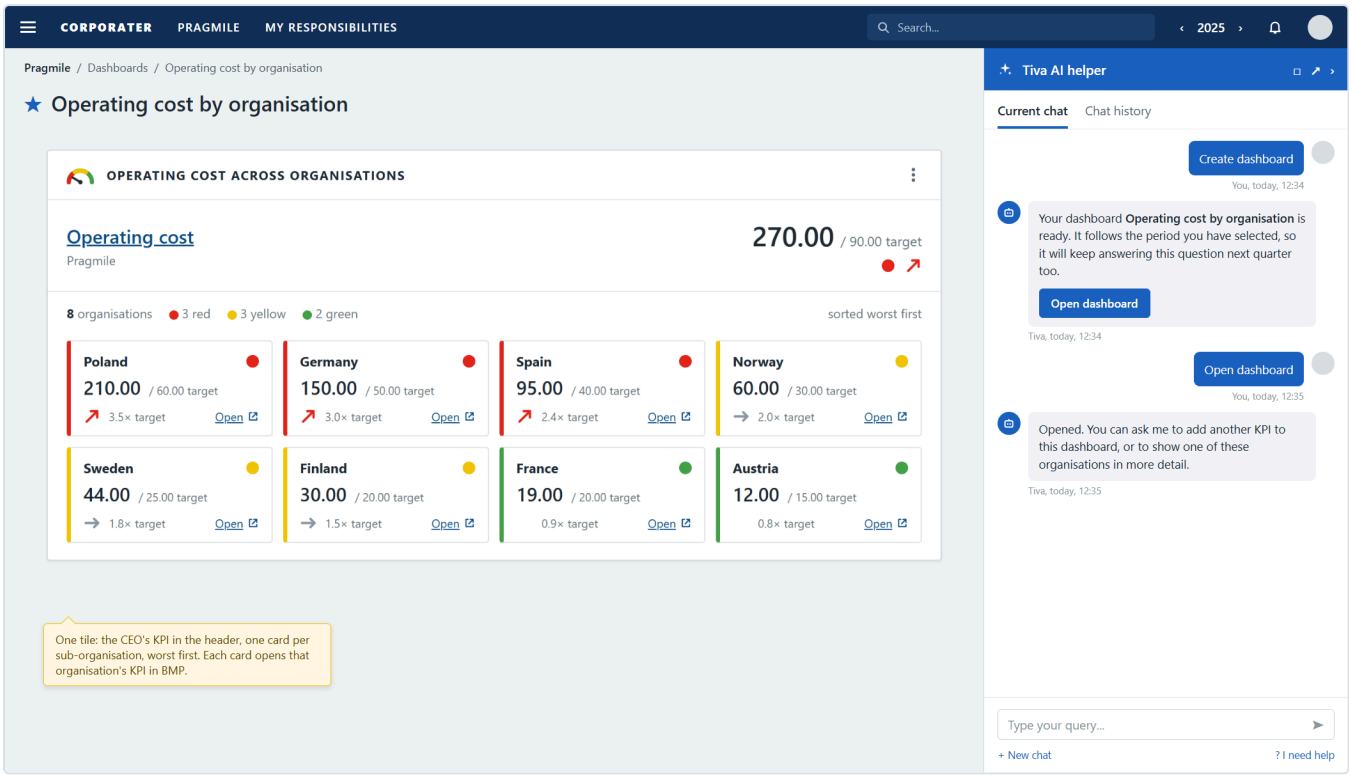
1. The top-level scorecard on 2025. Operating cost is red. The assistant button sits top right, as in the New BMP 2026 design: Figma, New BMP 2026, node 196-5622.



2. "Why is Operating cost red?" The assistant answers from the same data the dashboard will use: which sub-organisations are over target and by how much. Then it proposes a dashboard to track this over time. **Change since August:** the user approves before anything is created.



3. After "Create dashboard", progress streams into the chat. When done, the assistant shows an Open dashboard button. **Change since August:** the view does not switch by itself.



4. The dashboard: one tile. The CEO's KPI in the header with actual, target, status and trend; one card per sub-organisation, worst first; a summary line. Each card opens that organisation's KPI in BMP, on the same period. The chat stays open for the next question.

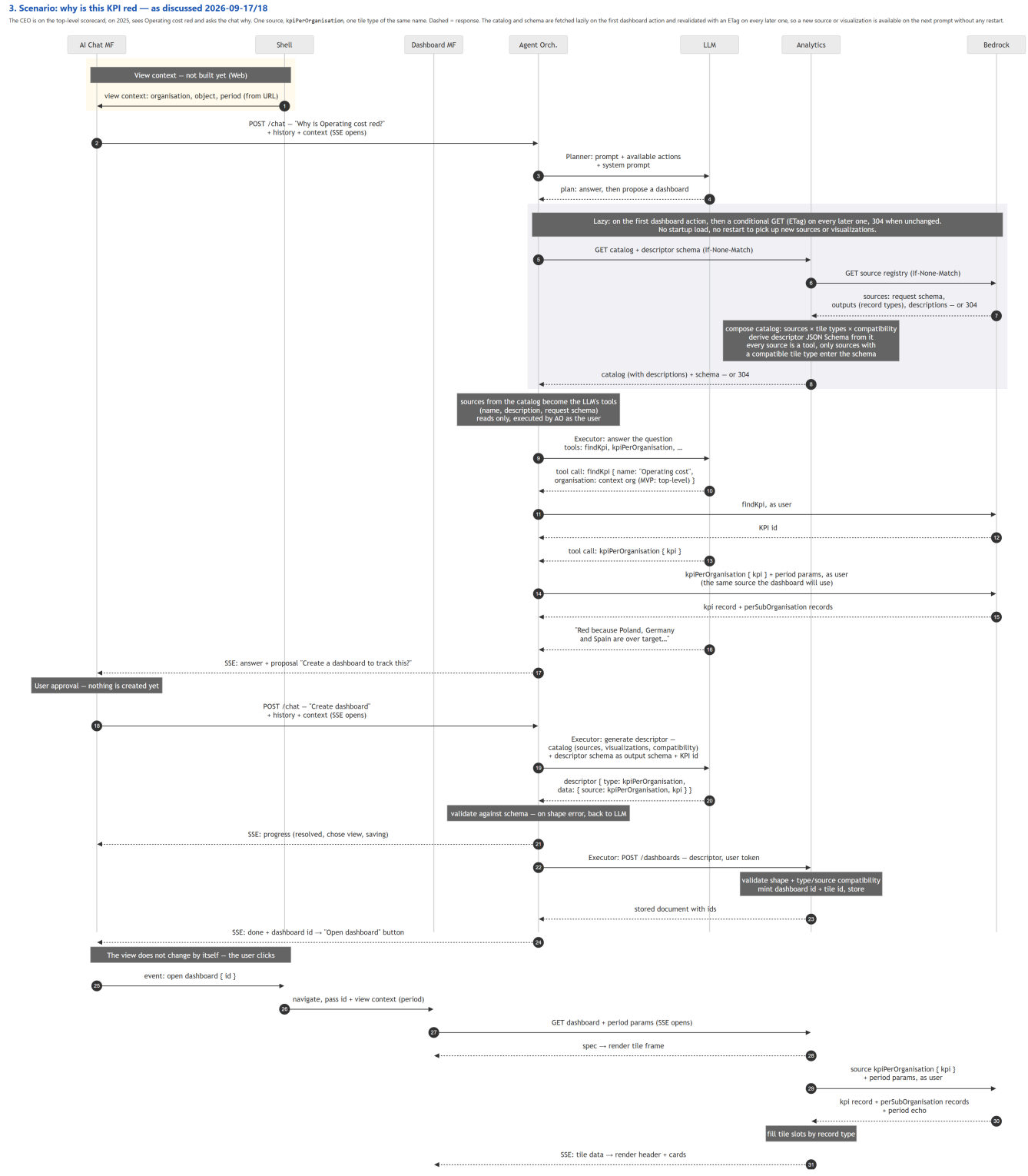
2. What changed since the August overview

- **Approval before creation.** The assistant answers first and proposes the dashboard. The create action runs only after the user says yes.
- **No automatic view change.** When the dashboard is ready the chat shows a button. On the click the chat emits an event on the Shell event bus and the Shell navigates. The "dashboard pending / open in loading state" steps are gone.
- **Sources instead of datasets.** A tile's data comes from a named *source*, a Bedrock endpoint with a published request schema and typed outputs. The generic query is one source (later). The MVP source is `kpiPerOrganisation`: one KPI id in, the same KPI on each direct sub-organisation out. Bedrock resolves "the same KPI" through the node the KPI objects share.
- **Sources are also the assistant's tools.** A source has a name, an owner-written description, a request schema and typed outputs, which is exactly a tool definition. Agent Orchestration turns the catalog into the LLM's tool list mechanically, so a new source in Bedrock is a new tool with no code change. Tools are reads, executed by Agent Orchestration as the user; the LLM still never writes. Every source is a tool; only sources with a compatible tile type appear in the dashboard schema. The answer in words and the tile use the same source call, so they cannot disagree.
- **Tile types are Analytics'.** The MVP tile type is also called `kpiPerOrganisation`, because the drawing knows the data's structure. A tile type declares slots; a source declares outputs; a tile is valid when the outputs fill the slots by record type and shape.
- **A catalog, fetched lazily.** Analytics composes a catalog of sources, tile types and their compatibility, with descriptions, and derives the descriptor JSON Schema from it. Agent Orchestration fetches it on the first dashboard action and revalidates with an ETag on every later one. Nothing is loaded at startup; a new visualization is available on the next prompt without a restart.
- **The view context travels with the request.** BMP already sends the current object and `selectedPeriod`, `selectedPeriodType`, `ytd` on every request. The Shell hands them to the chat and the dashboard, the dashboard passes them to Analytics, Analytics forwards them to Bedrock. A stored dashboard follows the viewer's period; the period is never stored. See section 7 on what happens if the Shell cannot provide the context in time.
- **Ids are minted by Analytics.** Dashboard and tile ids are assigned on create and returned. A writer sends no ids.
- **Aggregation moves to Bedrock**, into the query language, executed above the access filter. Not needed for the MVP; needed for the query source.

3. Components

Component	Role in the MVP
Shell (web-framework)	Hosts the microfrontends. Provides the view context to the chat and dashboard remotes: the object the user is looking at (here a scorecard, from which the organisation and its KPIs follow) and the period. Shows the assistant on every route. Navigates to the dashboard when the chat emits an open event on the Shell event bus. Web is looking into providing the context. **TBD**: whether the context arrives in time for the MVP; the bus mechanism.
AI Chat MF	Sends prompt, history and view context to Agent Orchestration over SSE. Renders the answer, the proposal with its buttons, progress, and the Open dashboard button.
Dashboard MF	Fetches the dashboard from Analytics with the period, renders the tile frame from the spec and the header and cards from streamed data. Links each card to the KPI in BMP with the period.
Agent Orchestration	Planner and PlanExecutor as before. Fetches the catalog and schema lazily and derives the LLM's tool list from the sources. Executes the LLM's tool calls (find the KPI, read `kpiPerOrganisation`) against Bedrock as the user, forwarding the period. On approval, generates the descriptor against the catalog and schema, validates it, posts it to Analytics. The LLM reads through tools and writes nothing.
Analytics	Owns the descriptor and the tile types. Composes the catalog from Bedrock's registry, exposes catalog and schema, validates and stores dashboards, mints ids, serves spec first and streams tile data, routes each `data` block to its source as the user, forwards the period.
Bedrock	Publishes a source registry. Serves `kpiPerOrganisation`: resolves counterparts by node, applies the user's access, computes status and trend for the request period. Returns the KPI record, one record per sub-organisation, the status definition and a period echo.
LLM	Plans, writes the answer in words, and emits the descriptor as structured output against the schema.

4. Sequence



1. View context from the Shell (not built). **2–4.** Prompt in, plan out: answer, then propose. **5–8.** Catalog and schema, lazily, with conditional GETs down the chain; 304 when unchanged. **9–16.** The sources become the LLM's tools. It finds the KPI by name on the context organisation, top-level for the MVP, then reads `kpiPerOrganisation`, the same source the dashboard will use; Agent Orchestration executes both calls as the user. The LLM writes the answer from the records. **17.** Answer plus proposal; the user decides. **18–24.** On approval: descriptor generated against catalog and schema, validated twice, stored with minted ids, progress

streamed. **25–26.** The chat shows a button; the Shell navigates only on click and passes id and period. **27–31.** Spec first, then the source call with the period, slots filled by record type, data streamed to the tile.

5. The documents on the wire

5.1 The descriptor the assistant writes

```
{
  "schemaVersion": 1,
  "title": "Operating cost by organisation",
  "tiles": [
    {
      "type": "kpiPerOrganisation",
      "title": "Operating cost across organisations",
      "data": {
        "source": "kpiPerOrganisation",
        "kpi": "2102456963588136215"
      }
    }
  ]
}
```

Four fields per tile. `type` is what is drawn, `data.source` is where the data comes from, and everything else inside `data` is the source's own request schema, passed to Bedrock verbatim. Analytics returns the same document with a dashboard `id` and a tile `id`. No period, no layout, no comparison: each returns as an additive change when a tile on screen needs it.

5.2 What the source answers


```
{
  "period": {
    "selectedPeriod": 1735686000000,
    "selectedPeriodType": "Y",
    "ytd": false,
    "label": "2025"
  },
  "kpi": {
    "id": "2102456963588136215",
    "name": "Operating cost",
    "organisation": {
      "id": "2102456963588136001",
      "name": "Pragmile"
    },
    "statuses": [
      { "id": "red",    "label": "Red",    "colour": "#e2231a" },
      { "id": "yellow", "label": "Yellow", "colour": "#f2c500" },
      { "id": "green",  "label": "Green",  "colour": "#43a047" }
    ],
    "actual": 270.0,
    "target": 90.0,
    "format": {
      "decimals": 2,
      "unit": "kNOK"
    },
    "status": "red",
    "trend": {
      "direction": "up",
      "improving": false
    }
  },
  "perSubOrganisation": [
    {
      "id": "2102456963588136301",
      "organisation": {
        "id": "2102456963588136002",
        "name": "Poland"
      },
      "actual": 210.0,
      "target": 60.0,
      "format": { "decimals": 2, "unit": "kNOK" },
      "status": "red",
      "trend": {
        "direction": "up",
        "improving": false
      }
    },
    {
      "id": "2102456963588136377",
      "organisation": {
        "id": "2102456963588136003",
        "name": "Norway"
      },
      "actual": 60.0,
      "target": 30.0,
```

```
      "format": { "decimals": 2, "unit": "kNOK" },
      "status": "yellow",
      "trend": {
        "direction": "flat",
        "improving": false
      }
    }
  ]
}
```

- Two outputs, `kpi` (one record) and `perSubOrganisation` (a list, direct sub-organisations only), plus which period the figures are for.
- Each record carries its number format, decimals and unit, so the tile shows figures the way BMP does and formats nothing on its own.
- The status definition comes once, ordered worst first, because BMP statuses are configurable and may have more than three zones. Records reference a zone by id. Sorting, tallies and the gauge follow the order, not the colours.
- Trend says whether it is an improvement, because the tile does not know the KPI's direction and Bedrock does.
- Children carry only what differs per organisation. Nothing derived is returned: counts, ratios, order and links are the frontend's.

5.3 The catalog, in outline

```
{
  "sources": [
    {
      "id": "kpiPerOrganisation",
      "description": "One KPI and the same KPI on each of its sub-organisations, with actual,
                      target, status and trend for the current period. Use when the question
                      is how one KPI is doing across the organisation.",
      "request": {
        "kpi": { "type": "string", "description": "Id of the KPI to start from" }
      },
      "outputs": [
        { "name": "kpi",                "shape": "record", "of": "KpiRecord" },
        { "name": "perSubOrganisation", "shape": "list",   "of": "KpiRecord" }
      ]
    }
  ],
  "tileTypes": [
    {
      "id": "kpiPerOrganisation",
      "description": "A header with one KPI's figures and status, then one card per
                      sub-organisation, worst first. Answers why a KPI has the status it has.",
      "slots": [
        { "name": "kpi",                "shape": "record", "of": "KpiRecord", "required": true },
        { "name": "perSubOrganisation", "shape": "list",   "of": "KpiRecord", "required": true }
      ],
      "compatibleSources": [
        { "source": "kpiPerOrganisation", "bind": null }
      ]
    }
  ]
}
```

The catalog is how a writer chooses, and it is also the assistant's tool list. Each source becomes a tool with the same name, description and request schema, so the LLM reads the source descriptions to pick the one that best answers the prompt, calls it to answer in words, and later picks it again as the tile's data source; then it reads the compatible tile types and their descriptions to pick how to show it. A lookup source with no visualization, such as finding a KPI by name, is a tool and nothing else. A person building a dashboard by hand sees the same descriptions as menus: pick a source, see which visualizations it can feed; or pick a visualization, see which sources fill it. Both come from one matrix, so the two writers cannot disagree about what goes together.

Bedrock writes the source and output descriptions, because it knows what "the same KPI" means. Analytics writes the tile type and slot descriptions, because it knows what they draw and when to pick them. One text per thing serves the prompt and the configurator alike, so the explanation and the validator cannot drift apart. The JSON Schema the LLM writes against is derived from this catalog, with the same descriptions in its `description` fields.

6. What each team builds for the MVP

Team	Work
Bedrock	Source registry endpoint. A `findKpi` lookup source (name and organisation in, KPI id out) and the `kpiPerOrganisation` endpoint: KPI id in, period from request parameters, parent record plus one record per direct sub-organisation out, each with its number format, plus the status definition and a period echo. "The same KPI" is resolved through the node the KPI objects share.
Analytics	Fetch the registry, compose catalog and schema, expose both with ETags. Descriptor v1 with `type`, `title`, `data.source`. Validate shape and compatibility, mint ids, store. Serve spec then stream tile data; route `data` by source; forward period parameters. Tile type `kpiPerOrganisation` with its slots and description.
Agent Orchestration	Lazy catalog and schema fetch with ETag. Derive the tool list from the sources and execute tool calls against Bedrock as the user, with the period. Actions: answer with tools, propose, on approval generate descriptor with schema as output schema, validate, post. Stream progress and the final button.
Web (Shell)	View context (current object and period) to the chat and dashboard remotes. Assistant button and panel on every route, not only the dashboard route. Shell event bus, and on the open event: navigate to the dashboard route with id and view context.
Dashboard MF	Fetch with period, render the `kpiPerOrganisation` tile: header, summary line, cards worst first, links with period.

7. Assumptions and open items

- **View context: both or neither.** The Shell either provides the object the user is looking at (a scorecard here, from which the organisation and its KPIs follow) together with the period, or it provides nothing yet. It is one problem, and Web is looking into it. This is not a nice-to-have: the assistant cannot know which "Operating cost" the user means without it, since every sub-organisation has a KPI of that name, and a dashboard cannot follow the viewer's period without it. We cannot go to production without the context. The only reason to run the MVP without it is to avoid being slowed by the dependency on Web; if we do, the MVP assumes the top-level organisation and Bedrock's default period, and the demo data must exist for that period.
- **TBD** Whether Web can deliver the view context in time for the MVP. This one is genuinely open; see the item above for what depends on it.
- **Chat panel everywhere in the Shell.** The assistant button and panel are available on every route, BMP pages included, as in the Figma design. Today the panel is shown only on the dashboard route. Web.
- **Navigation over the Shell event bus, as in August.** When the user clicks Open dashboard, the chat emits an event with the dashboard id and the Shell navigates to the dashboard route, passing id and view context to the dashboard remote. We assume Web provides the bus; if it cannot, this scenario has to be rethought, because the alternatives (a navigate callback on the bridge, a plain link) change how the chat keeps its state across the route change. **TBD** on Web's side: the bus mechanism, and whether URL parameters survive the navigation.
- **Decided for Bedrock's source.** `kpiPerOrganisation` takes the KPI id and walks one level down, direct sub-organisations only; deeper walks can come later as an additive change. Each record carries its number format (decimals, unit) so the tile formats figures as BMP does. Links to a KPI in BMP are built by the frontend from the record's id and the period.

- Later, not MVP: the `query` and `dataset` sources with aggregation in Bedrock; a generic `metric` tile and `bind`; layout (width, height); comparison; cross-filter; version history.

Mockups and diagram are illustrative. Figures are demo data. Source of this document: the Analytics design discussion of 16–18 Sep 2026.